AP-CWS: Predictive Cold-Start-Aware Serverless Workflow Scheduler

Nikhil Sharma, Raghav Arora, and Jashanpreet Singh
Department of Mechanical Engineering
Indian Institute of Technology (BHU), Varanasi, India

***Abstract*—Serverless platforms increasingly support applications composed of multiple functions connected through Directed Acyclic Graphs (DAGs). However, cold starts on the critical path can significantly affect workflow performance. When a function without an initialized container is invoked, the time required to initialize its execution environment delays its execution and, consequently, the dependent tasks. These delays can accumulate across a workflow, increasing tail latency and leading to Service Level Objective (SLO) violations.**

Existing schedulers generally treat warm container pool sizing and DAG-based task scheduling as separate problems. Container allocation is often driven primarily by function invocation frequency, without considering the impact of individual functions on overall workflow completion time. As a result, resources may be wasted on less critical functions, while latency-sensitive tasks remain exposed to initialization delays.

To address this limitation, this paper proposes AP-CWS, a scheduler that jointly considers warm container pool sizing and task placement. The proposed approach combines Holt's linear exponential smoothing, DAG-based topological slack budgeting, joint placement scoring, and Just-In-Time (JIT) prewarming of successor functions. To support high-throughput clusters, we further introduce Fast-AP-CWS, an optimized variant that employs memoized DAG metrics and power-of-two-choices node selection to achieve $O(1)$ dispatch time.

We evaluate the proposed scheduler through 324 simulation runs involving three workflow types and four different arrival patterns. The results show that AP-CWS achieves a P95 tail latency of 2.83 s, representing a 27.8% improvement over the baseline schedulers. Furthermore, it reduces critical-path cold starts to 0.63%, lowers SLO violations by 84.4%, and reduces overall cost by 4.6%.

***Index Terms*—Serverless computing, Function-as-a-Service (FaaS), Directed Acyclic Graph (DAG), cold start, tail latency, critical path, workflow scheduling, resource allocation, discrete-event simulation.**

I. Introduction

Serverless computing, or Function-as-a-Service (FaaS), has become a widely used approach for running event-driven cloud applications [1], [2]. Developers write their application logic as small, stateless functions, while the cloud provider takes care of the underlying infrastructure, auto-scaling, and fault tolerance. Since resources are allocated on demand, users are generally charged only for the time their functions execute. This makes serverless computing particularly useful for applications with bursty workloads.

In many applications, multiple serverless functions are connected through Directed Acyclic Graphs (DAGs) to form workflows [3]–[5]. However, these workflows are affected by unpredictable container initialization delays, commonly known as cold starts [1]. A cold start occurs when a request arrives but no initialized container is available to handle it. The platform must then prepare the execution environment by setting up a sandbox, retrieving the container image, and loading the runtime. This process can take around 0.5 to 2.5 seconds, compared to less than 200 ms for execution in a warm container.

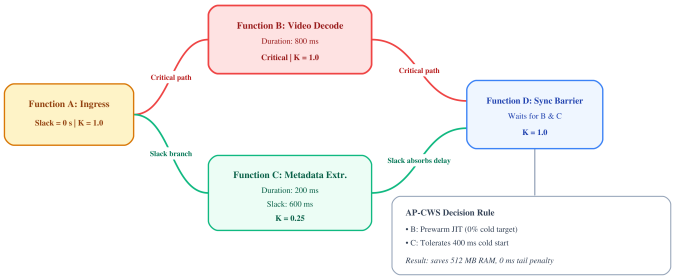


Fig. 1. Topological Slack and Criticality Differentiation in Serverless DAG Workflows. Function f_A lies on the zero-slack critical execution path where container initialization delays directly inflate overall workflow completion time. In contrast, parallel task f_B possesses execution slack, which absorbs cold-start latency without impacting workflow completion time.

In workflows with multiple dependent functions, a cold start can affect more than just the function being initialized. When a function on the critical path experiences a delay, its successor tasks must wait for its execution to finish. This can cause delays to propagate through the workflow, particularly when multiple functions along the same path experience cold starts. We refer to this effect as a *cascading cold start*.

However, not every function has the same impact on workflow completion time. As illustrated in Fig. 1, functions on parallel branches may have execution slack [6], which represents the additional time a task can wait without delaying the overall workflow. This means that cold-start delays on such functions can sometimes be absorbed without affecting the final completion time.

Current serverless platforms do not account for these differences in function criticality. Most rely on fixed keep-alive timers, retaining idle containers for 10 to 15 minutes [1]. While this helps reduce cold starts, it does not adapt to changing traffic patterns and can lead to unnecessary memory consumption during periods of low activity.

Recent research has explored predictive prewarming to address this problem. One notable approach is SSC by Pan et al. [7], which divides resource management into two stages. It first determines warm pool sizes based on function invocation frequency and then identifies the critical path using CPU and memory requirements.

However, this separation introduces several limitations:

1) Warm pools are sized uniformly across functions, po-

TABLE I
SYSTEM NOTATION, MATHEMATICAL SYMBOLS, AND OPERATIONAL DEFINITIONS

Symbol / Variable	Structural Category	Operational Definition & Mechanistic Role	Physical Dimension / Domain
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	Workflow Model	Directed Acyclic Graph of constituent function tasks $\mathcal{V}$ and precedence dependency edges $\mathcal{E}$	Topological graph structure
$f_i \in \mathcal{V}$	Workflow Task	Individual serverless function task instance ($i \in \{1, \dots, N\}$)	Discrete execution entity
$e_{ik} = (f_i, f_k)$	Dependency Edge	Precedence constraint requiring complete termination of parent f_i before dispatching child f_k	Precedence pair in $\mathcal{V} \times \mathcal{V}$
m_i, c_i	Task Resource	DRAM memory allocation requirement and normalized CPU core quota demanded by function f_i	Megabytes (MB), Cores
t_i^{base} ($\bar{t}_i$), s_i	Execution Profile	Nominal computational execution duration on baseline node ($v = 1.0$) and container startup cold delay	Seconds (s)
d_{ik}	State Transfer	Intermediate serialized application state data transferred across dependency edge (f_i, f_k)	Megabytes (MB)
EST_i, EFT_i	Topological Timing	Earliest Start Time and Earliest Finish Time computed via forward topological traversal of $\mathcal{G}$	Seconds (s)
LFT_i, LST_i	Topological Timing	Latest Finish Time and Latest Start Time computed via backward topological traversal from sink nodes	Seconds (s)
T^*	Critical Makespan	Theoretical minimum workflow completion time along the zero-slack critical execution path	Seconds (s)
Slack_i	Execution Slack	Spare delay buffer before downstream delay propagation ($\text{Slack}_i = LST_i - EST_i$)	Seconds (s)
K_i	Criticality Index	Dimensionless topological urgency index: $K_i = 1 - (\text{Slack}_i / T^*)$; sensitivity to startup delay	Real interval $[0, 1]$
$\mathcal{J}$	Compute Cluster	Heterogeneous worker nodes available for container placement and execution ($j \in \{1, \dots, M\}$)	Node set
M_j, C_j	Physical Capacity	Total physical DRAM memory and total hardware processing cores available on worker node j	MB, Physical Cores
v_j, p_j	Node Specifications	Hardware CPU speed scaling multiplier ($t_{ij} = t_i^{base}/v_j$) and active unit financial billing tariff	Multiplier, $/GB-hr
$\tau_{ik}^{net}(j, j')$	Network Latency	Inter-node data serialization and transmission delay (0 if co-located; $d_{ik}/B_{net} + \delta_{prop}$ if cross-node)	Seconds (s)
Δ	Planning Interval	Periodic offline control loop epoch for Holt demand forecasting and warm pool dimensioning	Seconds ($\Delta = 60$ s)
L_t, T_t	Demand State	Smoothed base level and trend slope components updated via Holt linear exponential smoothing	Invocations/s
α, β	Smoothing Weights	Moving level weight and trend updating coefficients ($\alpha = 0.20, \beta = 0.10$)	Real interval $[0, 1]$
$\hat{\lambda}_i$	Projected Demand	Forecasted invocation arrival rate for function f_i across subsequent planning window Δ	Invocations/s
p_i^{target}	Reliability Budget	Target allowable cold-start probability ceiling assigned to task f_i based on criticality K_i	Real interval $[0, 1]$
W_i^*, W_{ij}	Warm Pool Sizing	Target cluster-wide warm container quota and discrete warm container pool allocated on node j	Non-negative integers $\mathbb{Z}_{\geq 0}$
$x_{ij} \in \{0, 1\}$	Placement Indicator	Binary decision variable: 1 if task instance f_i is dispatched to worker node j, and 0 otherwise	Boolean binary
$y_{ij} \in \{0, 1\}$	Startup Status	Binary state flag: 0 if warm container is acquired on node j, and 1 if cold initialization is incurred	Boolean binary
S_{ij}	Dispatch Objective	Multi-factor cost-latency placement evaluation score quantifying suitability of node j for task f_i	Scalar value
μ, γ, η	Tuning Weights	Relative financial cost penalty weight ($\mu = 0.05$), memory headroom bonus ($\gamma = 0.10$), and criticality scaling factor ($\eta = 1.5$)	Scalar coefficients
κ	Prewarm Threshold	Criticality threshold triggering asynchronous Just-In-Time (JIT) successor container prewarming	Criticality threshold ($\kappa = 0.60$)
d	Sub-Sampling Size	Number of randomly sampled candidate nodes evaluated under Power-of-Two-Choices in Fast-AP-CWS	Integer ($d = 2 \ll \lvert\mathcal{J}\rvert$)
T_{P95}, T_{SLO}	Quality Metric	Measured 95th-percentile end-to-end workflow completion time and target contractual deadline	Seconds (s)

tentially wasting memory on tasks that can tolerate initialization delays.
2) Critical paths are computed using normal execution times, without accounting for how cold starts can alter task timing.
3) Node placement considers CPU and memory availability but does not account for the distribution of warm containers across nodes.

To address these limitations, this paper proposes AP-CWS, which brings demand forecasting, slack budgeting, warm pool sizing, and task placement together within a unified scheduling framework. The notation used throughout the paper is summarized in Table 1.

The main contributions of this work are as follows:

1) A criticality metric $K_i \in [0, 1]$, derived through forward and backward passes over the DAG [6].
2) A dual-loop architecture consisting of a 60-second planning loop based on Holt's linear exponential smoothing [8] and an event-driven dispatch loop.
3) A Just-In-Time (JIT) prewarming mechanism that initiates successor function preparation while the parent function is still executing.
4) Fast-AP-CWS, an optimized variant for large clusters that uses Power-of-Two-Choices with $d = 2$ to achieve $O(1)$ dispatch time [9].
5) An evaluation covering 324 simulation runs, along with stress tests to identify the performance limits of the proposed approach.

II. Related Work

Serverless cold-start techniques fall into three groups: container caching, predictive prewarming, and DAG scheduling. Table 2 has all the baselines we compare against.

A. Keep-Alive and Cache

Industry platforms use fixed keep-alive timers. Shahrad et al. [1] looked at real serverless workloads and found big swings in how often functions get called. Most platforms keep containers around for 10 minutes whereas this uses memory without thinking about the app structure. FaasCache by Fuerst and Sharma [11] shows that greedy caching works for independent functions, CASR by Chen et al. [10] uses TinyLFU [12] to balance recency and frequency. It helps leaf functions but hurts intermediate sync tasks. Hanaforoosh et al. [13] explored task aggregation using granularity trees to fuse small functions, but this requires static compilation changes. In a DAG, join tasks might not get called much but they're on the critical path.

The second group tries to predict when containers are needed. Nguyen et al. [14] use Model Predictive Control but the problem is it defers requests during traffic spikes. This adds queueing delays that break workflow deadlines. DROPS by Alquraan et al. [15] uses quantile tracking to size pools. It reserves capacity uniformly across all parts of the workflow so that when traffic spikes, it fills memory with idle containers and causes thrashing.

The third group looks at DAG scheduling while respecting execution precedence [3]–[5], [16]. Wukong by Carver et al. [5] focuses on locality. FaaSFlow by Li et al. [3] splits workflows into subgraphs to balance communication costs. Pheromone by Yu et al. [4] uses data-centric coordination to avoid bottlenecks. Zolfaghari et al. [16] look at placement in edge clusters.

Our work builds on SSC by Pan et al. [7]. SSC splits resource management into two steps. First it uses frequency gradients for prewarm counts, then it finds the critical path for CPU and memory.

This split doesn't work well as SSC sizes pools without checking if a function has slack. So memory goes to tasks that could tolerate cold starts. Also, SSC places tasks without

TABLE II
COMPARATIVE TAXONOMY OF EVALUATED SERVERLESS SCHEDULING BASELINES

Reference & Venue	Evaluated Baseline	Optimization Objective	Prewarming Strategy	DAG Structural Awareness	Task-Node Placement Coupling	Core Algorithmic Limitation
Shahrad et al., *USENIX ATC '20* [1]	Fixed 10-Min Keep-Alive	Minimize startup frequency	Fixed 10-minute idle container retention	None (Independent function invocations)	Decoupled (Default greedy bin-packing)	Keeps idle containers regardless of future demand; causes memory exhaustion during load spikes.
Pan et al., *IEEE T-SUSC '24* [7] (Base Paper)	SSC	Minimize dollar cost and makespan	Frequency gradient trend extrapolation	Critical path used only for CPU/RAM sizing	Decoupled (Sequential independent stages)	Sizes pools without topological slack; places critical tasks without checking warm container locality.
Chen et al., *FGCS '25* [10]	CASR	Reduce cold starts under cache limits	Frequency-recency caching (W-TinyLFU [12])	None (Local container eviction)	Decoupled (Host-local container cache)	Evicts infrequent intermediate join tasks; creates cascading tail delays at workflow barriers.
Nguyen et al., *MASCOTS '25* [14]	MPC	Minimize cold starts under memory bounds	Rolling-horizon queue deferral and prewarm	None (Applies to isolated function queues)	Decoupled (Centralized memory bounds)	Request deferral introduces artificial queueing delays that violate strict workflow deadlines.
Alquraan et al., *EuroSys '26* [15]	DROPS	Bound tail latency under strict SLOs	Quantile-based dynamic pool dimensioning	Weak (End-to-end deadline partitioning)	Decoupled (Cluster-wide reservation)	High-percentile sizing overprovisions idle warm pools; triggers severe memory contention under high concurrency.
Proposed Work	**AP-CWS & Fast-AP-CWS**	**Minimize P95 latency, cost, and SLO violations**	**Topological slack budgeting with JIT prewarming**	**Full DAG awareness (Dynamic K_i slack tracking)**	**Fully Coupled (Joint warmth, cost, and speed scoring)**	**Requires workflow graph definition and periodic matrix updates.**

checking if warm containers exist on the node. It often sends critical tasks to cold hosts. AP-CWS fixes this by combining slack budgeting, warm-aware placement, and JIT prewarming into one system.

III. System, Network, and Energy Node Models

A. Heterogeneous Serverless Cluster Architecture

We model a cluster managed by a central control plane with worker nodes $\mathcal{J} = \{1, 2, \ldots, M\}$. Each worker node $j \in \mathcal{J}$ has: 1) C_j: CPU cores. 2) M_j: RAM in MB. 3) v_j: Speed factor ($v_j = 1.2$ is 20% faster than baseline 1.0). 4) p_j: Cost per gigabyte-hour ($/GB-hr).

Each node runs a daemon for container lifecycles, keeping active memory for running functions and warm memory for idle containers.

B. Container Lifecycle and Execution State Machine

A function f_i needs m_i MB and c_i cores. When dispatched to node j, its container goes through these states: 1) *State 0 (Uninstantiated):* No container exists on node j. 2) *State 1 (Cold Initialization):* If no warm container is ready, the node cold starts by pulling the image, setting up cgroups, and launching the runtime, taking delay s_i. 3) *State 2 (Active Execution):* The container runs the logic. Runtime t_{ij} depends on base duration t_i^{base} and node speed v_j:

$$t_{ij} = \frac{t_i^{base}}{v_j} \tag{1}$$

4) *State 3 (Idle Warm Keep-Alive):* After finishing, it stays in an idle warm state, holding memory m_i. If called again before timeout, it runs instantly with zero delay. 5) *State 4 (Eviction):* If the timer expires or memory is needed, it shuts down and frees m_i.

C. Network Communication Model

Between parent f_i and child f_k, data size is d_{ik} MB. Network delay τ_{ik}^{net} between nodes j and j' is:

$$\tau_{ik}^{net}(j, j') = \begin{cases} 0, & \text{if } j = j' \\ \frac{d_{ik}}{B_{net}} + \delta_{prop}, & \text{if } j \neq j' \end{cases} \tag{2}$$

where B_{net} is bandwidth (10 Gbps) and δ_{prop} is propagation latency. Co-locating tasks on the same node cuts out transfer delay.

D. Energy and Resource Consumption Model

Energy consumption for node j over interval Δ includes static idle power, active execution power, startup power, and memory retention power:

$$\begin{aligned} E_j(\Delta) = & P_j^{idle} \cdot \Delta + \sum_{f_i \in \mathcal{A}_j} P_{ij}^{exec} \cdot t_{ij} \\ & + \sum_{f_i \in \mathcal{C}_j} P_{ij}^{cold} \cdot s_i + \sum_{f_i \in \mathcal{W}_j} P_j^{mem} \cdot m_i \cdot \Delta_i^{warm} \end{aligned} \tag{3}$$

where P_j^{idle} is static idle power, P_{ij}^{exec} is CPU power, P_{ij}^{cold} is startup burst power, and P_j^{mem} is memory power. Sets $\mathcal{A}_j$, $\mathcal{C}_j$, and $\mathcal{W}_j$ are active, cold, and warm tasks. Keeping warm containers uses memory power, but cold starts cause CPU spikes. Schedulers need to balance both.

IV. Problem Formulation

A. Workflow DAG Representation

We represent a serverless workflow application as a Directed Acyclic Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where: 1) $\mathcal{V} = \{f_1, f_2, \ldots, f_N\}$ is the set of N constituent serverless functions. 2) $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of directed edges representing execution precedence and data dependencies. An edge $(f_i, f_k) \in \mathcal{E}$ indicates that child function f_k cannot begin execution until parent function f_i completes and delivers its output data.

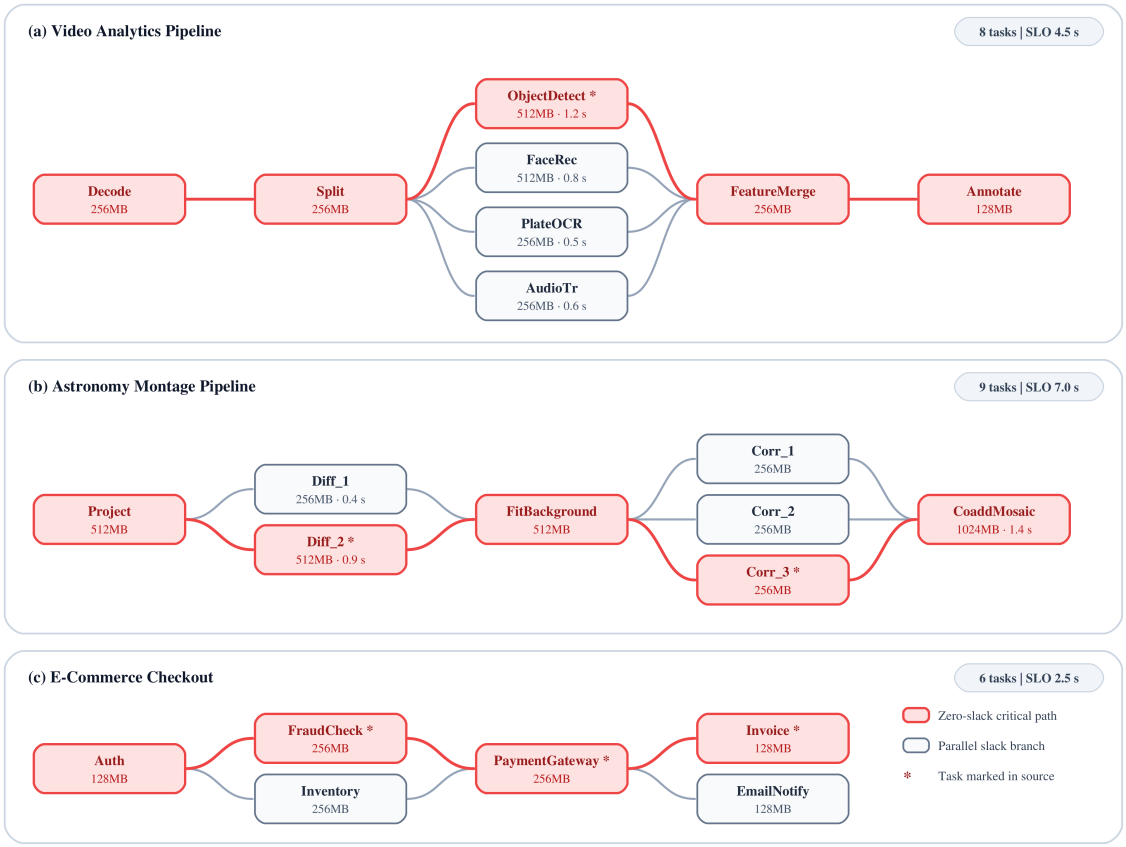


Fig. 2. Structural Topologies of Evaluated Serverless Workflow Benchmarks: (a) Video Analytics Pipeline (8 functions, depth 5, deadline 4.5 s), (b) Astronomical Montage Pipeline [17] (9 functions, depth 6, deadline 7.0 s), and (c) E-Commerce Transaction Pipeline (6 functions, depth 4, deadline 2.5 s). Asterisks and bold red paths mark zero-slack critical execution sequences.

Each workflow has defined entry functions $\mathcal{V}_{entry} = \{f \in \mathcal{V} \mid \text{parents}(f) = \emptyset\}$ and exit functions $\mathcal{V}_{exit} = \{f \in \mathcal{V} \mid \text{children}(f) = \emptyset\}$. The application is governed by an end-to-end Service Level Objective deadline, denoted by T_{SLO}. Fig. 2 illustrates the structural topologies and deadline configurations of the evaluated serverless workflow benchmark pipelines.

B. Topological Timing Analysis and Slack Budgeting

To quantify execution flexibility across the graph, the scheduler performs forward and backward topological passes based on classical list scheduling principles [6]: 1) *Earliest Start Time (EST):* The earliest time a function can start executing, assuming all predecessor tasks execute without delay:

$$EST(f_i) = \begin{cases} 0, & \text{if } f_i \in \mathcal{V}_{entry} \\ \max\limits_{f_p \in \text{par}(f_i)} (EST(f_p) + \bar{t}_p), & \text{otherwise} \end{cases} \tag{4}$$

where $\bar{t}_p$ is the expected execution duration of parent task f_p on a baseline node, and $\text{par}(f_i)$ denotes the set of immediate parent functions. 2) *Earliest Finish Time (EFT):*

$$EFT(f_i) = EST(f_i) + \bar{t}_i \tag{5}$$

3) *Workflow Minimum Makespan (T^*):*

$$T^* = \max_{f_e \in \mathcal{V}_{exit}} EFT(f_e) \tag{6}$$

4) *Latest Finish Time (LFT):* The last moment a function can finish without delaying the entire workflow past T^*:

$$LFT(f_i) = \begin{cases} T^*, & \text{if } f_i \in \mathcal{V}_{exit} \\ \min\limits_{f_c \in \text{ch}(f_i)} (LST(f_c)), & \text{otherwise} \end{cases} \tag{7}$$

where $\text{ch}(f_i)$ represents immediate child tasks. 5) *Latest Start Time (LST):*

$$LST(f_i) = LFT(f_i) - \bar{t}_i \tag{8}$$

6) *Topological Slack (Slack_i):* Spare execution buffer for function f_i:

$$\text{Slack}_i = LST(f_i) - EST(f_i) \tag{9}$$

Tasks with $\text{Slack}_i = 0$ constitute the critical execution path of the workflow.

C. Topological Criticality Formulation

For each function f_i, we define the normalized criticality index K_i as:

$$K_i = 1 - \frac{\text{Slack}_i}{T^*} \tag{10}$$

Because $0 \leq \text{Slack}_i \leq T^*$, the criticality index is strictly bounded: $0 \leq K_i \leq 1$. 1) When $K_i = 1.0$: Function f_i has zero slack and is directly on the critical path. Any cold start

on f_i directly increases overall workflow completion time. 2) When $K_i \to 0$: Function f_i has substantial execution slack. A cold start on f_i will be absorbed by the slack buffer, having zero impact on end-to-end latency.

D. Decision Variables

The scheduling system controls two primary sets of decision variables: 1) $W_{ij} \in \mathbb{Z}_{\geq 0}$: Number of warm container instances for function f_i on node j during planning epoch Δ. 2) $x_{ij} \in \{0, 1\}$: Binary placement indicator, where $x_{ij} = 1$ if task instance f_i is dispatched to node j, and 0 otherwise. 3) $y_{ij} \in \{0, 1\}$: Binary startup indicator, where $y_{ij} = 0$ if task f_i finds an idle warm container on node j, and $y_{ij} = 1$ if it incurs a cold start.

E. Optimization Objectives and Constraints

The scheduling problem seeks to optimize three conflicting criteria: tail latency, cold-start rate, and monetary resource cost. 1) *P95 Workflow Completion Time (T_{P95}):*

$$\min \quad T_{P95}(\mathcal{G}) \tag{11}$$

where $T(\mathcal{G}) = \max_{f_e \in \mathcal{V}_{exit}} (t_e^{start} + t_{ej} + y_{ej} \cdot s_e)$. 2) *Critical-Path Cold-Start Rate (R_{crit}):*

$$\min \quad R_{crit} = \frac{\sum_{f_i \in \mathcal{V}} K_i \cdot y_{ij}}{\sum_{f_i \in \mathcal{V}} K_i} \tag{12}$$

3) *Total Monetary Cost (C_{total}):*

$$\begin{aligned} \min \quad C_{total} = \sum_{j \in \mathcal{J}} p_j \Bigg(& \sum_{f_i \in \mathcal{V}} x_{ij} \frac{m_i t_{ij}}{1024 \cdot 3600} \\ & + \sum_{f_i \in \mathcal{V}} W_{ij} \frac{m_i \Delta}{1024 \cdot 3600} \Bigg) \end{aligned} \tag{13}$$

Subject to the following operational constraints: 1) *Precedence Constraint:* For every edge $(f_i, f_k) \in \mathcal{E}$:

$$t_k^{start} \geq t_i^{finish} + \tau_{ik}^{net}(j, j'), \quad \forall (f_i, f_k) \in \mathcal{E} \tag{14}$$

2) *Node Memory Limit:* At any point in time t:

$$\sum_{f_i \in \mathcal{A}_j(t)} m_i + \sum_{f_i \in \mathcal{V}} W_{ij} \cdot m_i \leq M_j, \quad \forall j \in \mathcal{J} \tag{15}$$

3) *Node CPU Capacity Constraint:* Active CPU utilization on node j must not exceed total physical cores C_j:

$$\sum_{f_i \in \mathcal{A}_j(t)} c_i \leq C_j, \quad \forall j \in \mathcal{J} \tag{16}$$

4) *Unique Assignment Constraint:* Every task invocation must be assigned to exactly one worker node:

$$\sum_{j \in \mathcal{J}} x_{ij} = 1, \quad \forall f_i \in \mathcal{V} \tag{17}$$

5) *SLO Compliance Constraint:* Total workflow completion time should satisfy the target deadline:

$$T(\mathcal{G}) \leq T_{SLO} \tag{18}$$

Because exact optimization of this NP-hard problem is computationally intractable under sub-second latency constraints, we need an efficient heuristic. Sections V-A and V-B cover our dual-loop approach.

V. Proposed Approach

AP-CWS (Aware-Predictive Cold-Start Workflow Scheduler) is a scheduling framework for serverless DAG workflows. It runs two cooperating control loops: 1) *Periodic Offline Planning Loop:* Runs every $\Delta = 60$ s to forecast invocation rates, compute slack budgets, and set warm pool sizes. 2) *Event-Driven Online Dispatch Loop:* Runs on every invocation to score candidate nodes and trigger JIT prewarming for critical successors.

We also built Fast-AP-CWS for high-traffic clusters. Fig. 3 shows the overall two-phase architecture.

A. Periodic Planning Loop: Slack Budgeting and Warm Pool Sizing

The planning loop runs every Δ seconds to figure out how many warm containers each function needs.

1) Demand Forecasting via Holt Linear Exponential Smoothing

Traffic patterns have short-term trends and daily shifts. Moving averages react too slowly. Autoregressive models are too heavy so we use Holt linear exponential smoothing [8] to estimate the arrival rate $\hat{\lambda}_i$ for each $f_i \in \mathcal{V}$.

Let Y_t be the invocation count during period t. The forecast updates level L_t and trend T_t:

$$L_t = \alpha Y_t + (1 - \alpha)(L_{t-1} + T_{t-1}) \tag{19}$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1} \tag{20}$$

where $\alpha \in [0, 1]$ is the smoothing factor and $\beta \in [0, 1]$ is the trend factor. We set $\alpha = 0.2$ and $\beta = 0.1$. The predicted rate for the next period is:

$$\hat{\lambda}_i = \frac{\max(0, L_t + T_t)}{\Delta} \tag{21}$$

2) Topological Criticality and Cold-Start Probability Budgeting

Slack tells us how much delay a function can handle. Using T^* and $\text{Slack}_i = LST(f_i) - EST(f_i)$, we get $K_i = 1 - (\text{Slack}_i / T^*)$.

Other schedulers give all functions the same warm pool allocation. AP-CWS instead maps criticality to a cold-start probability budget p_i^{target}:

$$p_i^{target} = (1 - K_i) \cdot p_{max} + \epsilon \tag{22}$$

where $p_{max} = 0.25$ is the max cold-start probability for slack tasks, and $\epsilon = 0.02$ is the baseline floor. 1) When $K_i = 1.0$ (critical path), $p_i^{target} = \epsilon = 0.02$, a strict 2% ceiling. 2) When $K_i \to 0$ (parallel branch), $p_i^{target} \approx 0.25$, because slack absorbs any delays.

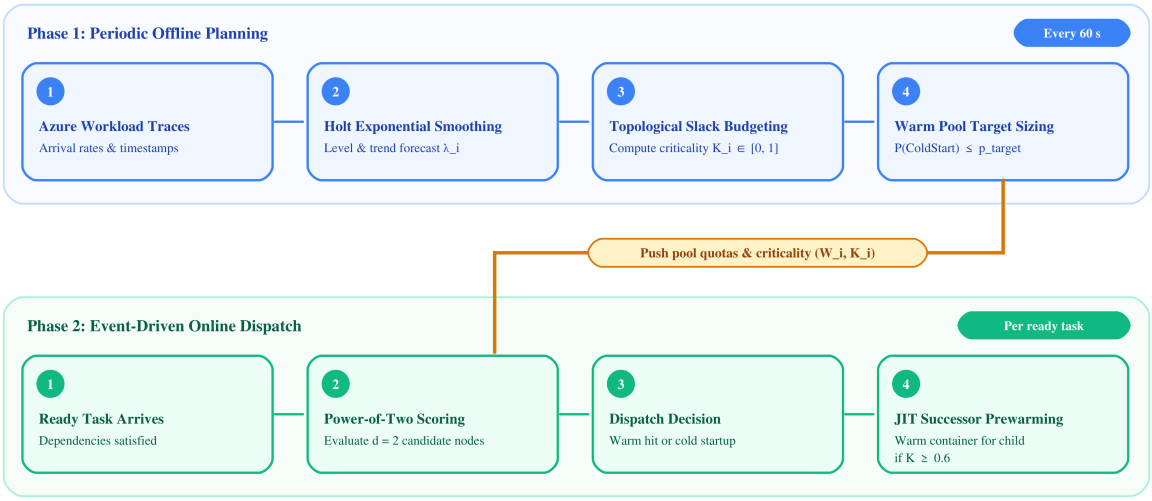


Fig. 3. AP-CWS Two-Phase Scheduling Architecture. The planning loop (left) updates demand forecasts via Holt smoothing and sets pool sizes based on slack budgets then the dispatch loop (right) places tasks using joint scoring and triggers JIT prewarming.

3) Analytic Pool Sizing Inversion

Cold-start probability versus pool size W_i follows exponential decay:

$$P(W_i) = \min\left(1.0, \alpha \cdot \exp(-\beta \cdot W_i)\right) \tag{23}$$

where $\alpha = 1.2$ and $\beta = 0.35$. Inverting this to satisfy $P(W_i) \leq p_i^{target}$:

$$W_i^* = \left\lceil -\frac{1}{\beta} \ln\left(\frac{p_i^{target}}{\alpha}\right) \right\rceil \tag{24}$$

The result gets clamped to $[W_{min}, W_{max}]$. Critical tasks with $K_i \geq 0.8$ get $W_{min} = 1$ so at least one warm container stays alive.

4) Cluster Warm Pool Placement

Once W_i is set, containers are packed onto nodes highest-criticality first, targeting nodes with bigger v_j and free memory $M_j^{free} \geq m_i$.

B. Event-Driven Dispatch Loop: Joint Scoring and JIT Prewarming

1) Multi-Factor Placement Scoring Function

Each candidate node gets a placement score S_{ij}:

$$\begin{aligned} S_{ij} =& (1 + \eta K_i)\left[y_{ij}\left(\frac{s_i}{v_j}\right) + q_j + \frac{t_i^{base}}{v_j}\right] \\ &+ \mu c_{ij} - \gamma\left(\frac{M_j^{free}}{M_j}\right) \end{aligned} \tag{25}$$

where $y_{ij} \in \{0, 1\}$ is 0 if warm and 1 if cold, s_i/v_j is startup time, t_i^{base}/v_j is run time, $q_j = 0.2 \times (1 - M_j^{free}/M_j)$ is queue pressure, $c_{ij} = p_j/0.05$ is normalized cost, $\mu = 0.05$, $\gamma = 0.10$, and $\eta = 1.5$.

The factor $(1 + \eta K_i)$ with $\eta = 1.5$ makes critical tasks care more about delays. When $K_i = 1.0$, penalties scale by 2.5, pushing the scheduler toward warm, fast nodes even if they cost more. When $K_i \to 0$, the weight stays near 1.0 so cost matters more.

Pick the best node:

$$j^* = \arg \min_{j \in \mathcal{J}, M_j^{free} \geq m_i} S_{ij} \tag{26}$$

2) Proactive Just-In-Time (JIT) Successor Prewarming

When f_i starts on j^*, the scheduler looks at its children $f_k \in \text{children}(f_i)$. If $K_k \geq \kappa$ ($\kappa = 0.60$) and j^* has no warm container for f_k, it sends an async prewarm signal. Parent execution overlaps with child startup:

$$t_{overlap} = \min(t_{ij^*}, s_k) \tag{27}$$

So the effective startup drops from s_k to $\max(0, s_k - t_{ij^*})$. When the parent finishes, child f_k is ready with zero network delay ($\tau_{ik}^{net} = 0$).

C. Operational Modes: AP-CWS vs. Fast-AP-CWS

Fig. 4 shows the two modes for different cluster sizes.

1) Standard AP-CWS: Global Optimality for Static Workloads

Standard AP-CWS checks all $|\mathcal{J}|$ nodes and picks the best one. This minimizes fragmentation and cost. Search overhead is only $41.4\,\mu$s on 128 nodes, under 0.05% of task time.

2) Fast-AP-CWS: Bounded Overhead for Hyperscale Workloads

At thousands of requests per second, linear scanning becomes a bottleneck. Fast-AP-CWS fixes this: 1) *Static DAG Criticality Memoization ($O(1)$ Lookup):* Precomputes EST, LST, K_i once at workflow registration and caches in a hash table. 2) *Power-of-Two-Choices Sub-Sampling ($O(1)$ Node Selection):* Checks for a warm container first; if none, randomly picks $d = 2$ nodes and uses the lower-scoring one.

Theory from [9] shows $d = 2$ cuts max queue length from $O(\ln|\mathcal{J}|/\ln\ln|\mathcal{J}|)$ to $O(\ln\ln|\mathcal{J}|)$. Fast-AP-CWS cuts dispatch from $41.4\,\mu$s to $13.9\,\mu$s on 128 nodes (3.0x faster) and handles over 71,000 dispatches per second.

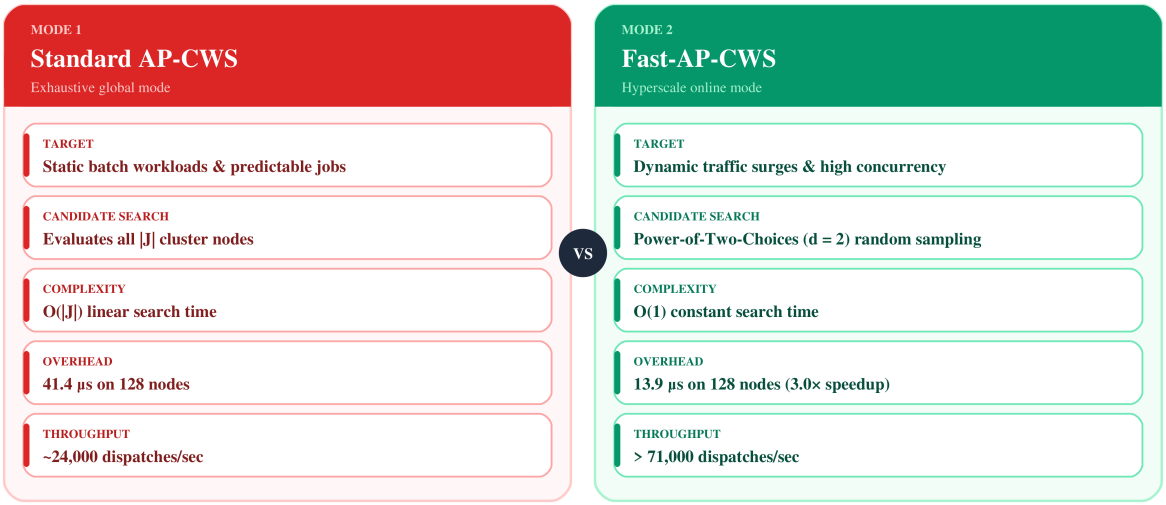


Fig. 4. AP-CWS Dual Modes: Standard AP-CWS for full global search in static workloads versus Fast-AP-CWS for sub-millisecond dispatch in hyperscale clusters.

TABLE III
HETEROGENEOUS CLUSTER WORKER NODE CONFIGURATIONS

Node ID	Instance Family	RAM (M_j)	CPUs	Speed	Cost Rate
Nodes 0–3	Compute-Opt. (c5)	8,192 MB	4.0	1.40×	$0.068/GB-h
Nodes 4–7	General-Purp. (m5)	16,384 MB	4.0	1.00×	$0.048/GB-h
Nodes 8–11	Memory-Opt. (r5)	16,384 MB	2.0	0.90×	$0.040/GB-h
Nodes 12–15	Burstable (t3)	4,096 MB	2.0	0.80×	$0.024/GB-h

VI. Result Analysis

A. Simulation Environment and Experimental Setup

We built a discrete-event simulator using SimPy, following the approach used by SimFaaS [19] and faas-sim [18]. It models container lifecycles, memory queues, concurrency, network delays, and billing.

1) Cluster Node Topology

The simulation cluster comprises 16 heterogeneous worker nodes modeled after commercial cloud instance families (Table III), providing 176 GB aggregate memory and 48 CPU cores. Cluster interconnect bandwidth is $B_{net} = 10$ Gbps with $\delta_{prop} = 2.0$ ms.

2) Benchmark Workflow Topologies

Three real-world DAG workflows were evaluated: 1) *Video Analytics Pipeline (8 Functions):* Processes camera feeds with 'Decode' (256 MB, 0.4 s), 'Split' (256 MB, 0.3 s), parallel analysis 'ObjectDetect' (512 MB, 1.2 s, critical), 'FaceRec' (512 MB, 0.8 s), 'PlateOCR' (256 MB, 0.5 s), and 'AudioTr' (256 MB, 0.6 s), synchronizing at 'FeatureMerge' (256 MB, 0.3 s) and concluding at 'Annotate' (128 MB, 0.2 s). $T_{SLO} = 4.5$ s. 2) *Montage Astronomical Workflow (9 Functions):* Reprojection 'Project' (512 MB, 0.8 s), background 'Diff_1' (256 MB, 0.4 s) and 'Diff_2' (512 MB, 0.9 s), fitting 'FitBackground' (512 MB, 1.1 s), corrections 'Corr_1'–'Corr_3' (256 MB, 0.5 s), and stacking 'CoaddMosaic' (1024 MB, 1.4 s) [17]. Critical path: 'Project' → 'Diff_2' → 'FitBackground' → 'Corr_3' → 'CoaddMosaic'. $T_{SLO} = 7.0$ s. 3) *E-Commerce Transaction Pipeline (6 Functions):* Authentication 'Auth' (128 MB, 0.2 s), 'FraudCheck' (256 MB, 0.5 s), inventory 'InventoryCheck' (256 MB, 0.3 s), payment 'PaymentGateway' (256 MB, 0.6 s), invoice 'InvoiceService' (128 MB, 0.2 s), and email 'NotificationEmail' (128 MB, 0.1 s). Critical path: 'Auth' → 'FraudCheck' → 'PaymentGateway' → 'InvoiceService'. $T_{SLO} = 2.5$ s.

3) Workload Arrival Patterns

Workload arrival intervals were generated using Azure Functions trace patterns [1]: Stable ($\lambda = 2.0$ req/s Poisson), Rising (1.0 to 6.0 req/s ramp), Falling (6.0 to 1.0 req/s ramp), and Bursty (bimodal 1.0 to 8.0 req/s surges). Each experiment ran for 300 s simulated time across multiple seeds, totaling 324 benchmark runs.

B. Performance Analysis and Evaluation

1) Tail Latency and Cold-Start Mitigation Analysis

Table IV shows AP-CWS gets a P95 tail latency of 2.83 s across all workflows which is 27.8% better than SSC (3.92 s) and 45.4% better than DROPS (5.18 s).

Fig. 5 shows these improvements broken down by workflow.

The biggest gain comes from cutting cold starts on the critical path. AP-CWS keeps critical-path cold starts at 0.63%, vs 2.34% for SSC, 2.95% for CASR, and 2.35% for DROPS. SSC sizes pools based on call frequency, ignoring the DAG, so critical-path containers get evicted. When one cold starts, its 1.2 s delay stalls everything downstream. AP-CWS starts successor containers while the parent is still running. By the time the parent finishes, the child is ready for execution.

Fig. 6 compares overall vs critical-path cold-start rates.

2) Workload-Specific Breakdown

Table V details performance disaggregated across the three benchmark workflows. In the Scientific Montage workflow, DAG depth is high (6 sequential stages), and individual

TABLE IV
CROSS-WORKFLOW PERFORMANCE COMPARISON (MEAN ± STANDARD DEVIATION ACROSS 324 RUNS)

Scheduling Policy	Mean Lat. (s)	Median P50 (s)	Tail P95 (s)	Cold Start %	Crit. Cold %	SLO Viol. %	5-Min Cost ($)	Idle RAM (GB-hr)
AP-CWS (Proposed)	**2.41 ± 1.18**	**2.33 ± 1.15**	**2.83 ± 1.55**	**0.58% ± 0.32%**	**0.63% ± 0.34%**	**0.46% ± 0.31%**	**$0.0390 ± 0.002**	3.52 ± 0.11
SSC (Pan 2024) [7]	2.83 ± 1.42	2.61 ± 1.34	3.92 ± 3.10	2.26% ± 1.12%	2.34% ± 1.18%	3.16% ± 1.45%	$0.0409 ± 0.003	3.27 ± 0.14
CASR (Chen 2025) [10]	3.36 ± 1.84	3.18 ± 1.76	4.61 ± 3.06	2.91% ± 1.48%	2.95% ± 1.51%	4.92% ± 1.88%	$0.0321 ± 0.002	1.76 ± 0.09
MPC (Nguyen 2025) [14]	3.12 ± 1.62	2.86 ± 1.54	4.62 ± 3.75	2.19% ± 1.05%	2.21% ± 1.08%	3.63% ± 1.52%	$0.0329 ± 0.002	2.13 ± 0.10
DROPS (EuroSys 2026) [15]	3.68 ± 1.95	3.51 ± 1.88	5.18 ± 3.92	2.29% ± 1.14%	2.35% ± 1.19%	4.51% ± 1.74%	$0.0466 ± 0.004	3.62 ± 0.15
Fixed 10-Min Keep-Alive [1]	3.37 ± 1.81	3.18 ± 1.72	4.68 ± 3.77	2.23% ± 1.08%	2.29% ± 1.11%	3.70% ± 1.60%	$0.0344 ± 0.002	2.12 ± 0.10

TABLE V
PER-WORKFLOW PERFORMANCE DISAGGREGATION ACROSS SCHEDULING BASELINES

Scheduling Policy	Mean Lat. (s)	Tail P95 (s)	Overall Cold %	Crit. Cold %	SLO Viol. %	5-Min Cost ($)
Video Analytics Pipeline (8 Tasks, $T_{SLO} = 4.5$ s)						
AP-CWS (Proposed)	**1.86**	**2.08**	**0.49%**	**0.37%**	**0.13%**	**$0.0366**
SSC (Pan 2024)	2.09	2.34	1.80%	1.58%	1.81%	$0.0399
CASR (Chen 2025)	2.52	3.17	2.06%	1.97%	2.62%	$0.0284
MPC (Nguyen 2025)	2.26	3.00	2.00%	1.87%	2.73%	$0.0275
DROPS (EuroSys 2026)	2.85	3.36	1.96%	1.82%	2.58%	$0.0423
Fixed 10-Min Keep-Alive	2.54	3.18	2.04%	1.94%	2.62%	$0.0290
Scientific Montage Pipeline (9 Tasks, $T_{SLO} = 7.0$ s)						
AP-CWS (Proposed)	**3.89**	**4.84**	**1.01%**	**1.17%**	**1.09%**	**$0.0488**
SSC (Pan 2024)	4.85	7.67	3.43%	3.81%	5.15%	$0.0558
CASR (Chen 2025)	5.69	8.44	4.98%	5.06%	9.42%	$0.0517
MPC (Nguyen 2025)	5.34	8.50	2.91%	3.05%	5.48%	$0.0555
DROPS (EuroSys 2026)	6.00	9.55	3.30%	3.48%	8.04%	$0.0627
Fixed 10-Min Keep-Alive	5.71	8.64	2.94%	3.10%	5.74%	$0.0578
E-Commerce Transaction Pipeline (6 Tasks, $T_{SLO} = 2.5$ s)						
AP-CWS (Proposed)	**1.47**	**1.57**	**0.24%**	**0.34%**	**0.17%**	**$0.0317**
SSC (Pan 2024)	1.54	1.77	1.56%	1.63%	2.53%	$0.0270
CASR (Chen 2025)	1.87	2.23	1.70%	1.83%	2.73%	$0.0163
MPC (Nguyen 2025)	1.77	2.37	1.65%	1.72%	2.68%	$0.0157
DROPS (EuroSys 2026)	2.20	2.64	1.60%	1.75%	2.92%	$0.0347
Fixed 10-Min Keep-Alive	1.87	2.23	1.70%	1.83%	2.73%	$0.0163

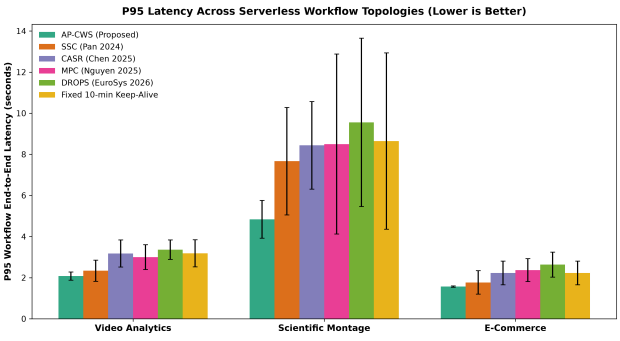


Fig. 5. P95 Tail Latency by Workflow (Lower is Better). AP-CWS gets 2.08 s in Video Analytics, 4.84 s in Scientific Montage, and 1.57 s in E-Commerce. Baselines suffer from critical-path cold starts.

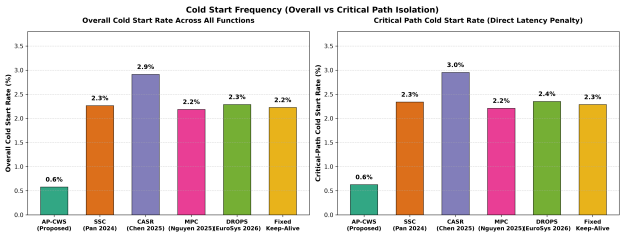


Fig. 6. Cold-Start Rate Comparison: Overall Workflow Tasks (left) versus Critical-Path Tasks (right). While baseline schedulers experience uniform cold-start penalties across both categories, AP-CWS selectively shields critical tasks, driving critical-path cold starts down to 0.63% through topological slack budgeting and proactive prewarming.

tasks require up to 1024 MB of memory. In this topology, uncoordinated placement causes severe pipeline stalls. SSC suffers a P95 latency of 7.67 s, violating the 7.0 s SLO in 5.15% of invocations. AP-CWS reduces P95 latency to 4.84 s (a 36.8% improvement) and suppresses SLO violations to 1.09%. In the E-Commerce pipeline, AP-CWS achieves a P95 latency of 1.57 s, eliminating tail variance compared to DROPS (2.64 s) and CASR (2.23 s).

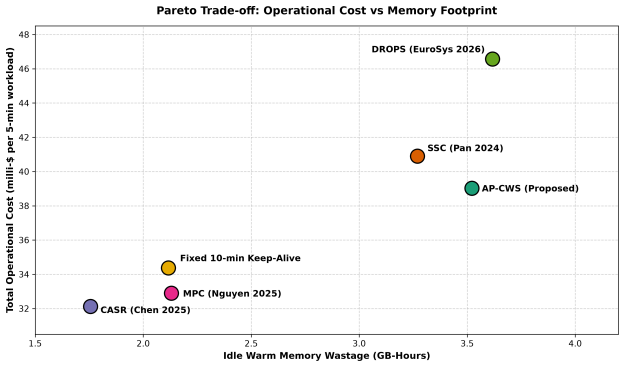


Fig. 7. Operational Dollar Cost versus Idle Memory Footprint Trade-off. Policies in the bottom-left quadrant minimize both metrics. Fixed Keep-Alive and CASR achieve low idle memory by evicting idle containers aggressively, but incur severe tail-latency penalties. DROPS inflates both cost and idle memory due to unconstrained quantile buffering. AP-CWS achieves lowest tail latency at competitive cost ($0.0390 per 5-minute window).

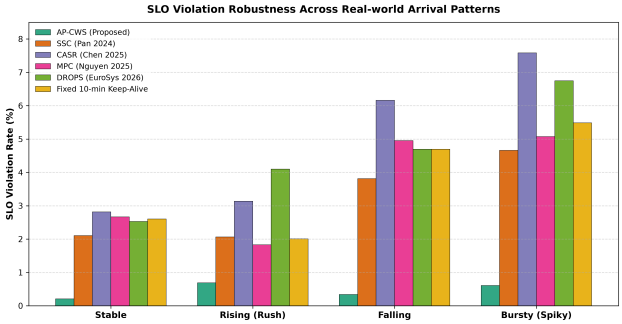


Fig. 8. Service-Level Objective (SLO) Violation Rates Across Evaluated Schedulers. AP-CWS reduces SLO violations to 0.46% (an 84.4% reduction relative to SSC at 3.16% and a 90.7% reduction relative to CASR at 4.92%) by eliminating synchronization stalls on critical-path functions.

Figure 7 evaluates the trade-off between operational dollar cost and idle memory footprint, illustrating the Pareto efficiency of AP-CWS relative to competing baselines.

Corresponding end-to-end contractual compliance is illustrated in Figure 8 across all arrival regimes.

3) Scalability Stress Testing and the 12 req/s Cascading Latency Cliff

We swept an 8-node cluster (64 GB total memory) from 1.0 req/s to 32.0 req/s with the Video Analytics workflow to identify operational breaking points. Empirical results are shown in Table VI.

The stress test reveals two distinct operational breaking points: 1) *The 12 req/s Cascading Latency Cliff (Algorithmic Failure in Baselines):* At $\lambda = 12.0$ req/s, incoming load reaches 96 functions per second. Under SSC, P95 latency spikes dramatically from 6.56 s to 14.39 s (a 2.2x increase), and SLO violations surge to 14.07%. DROPS and Fixed Keep-Alive exhibit identical degradation, with latencies inflating to 14.88 s. This breakdown occurs because decoupled schedulers do not coordinate container retention with task placement. At 12 req/s, host physical memory becomes saturated with idle warm containers. When a critical task arrives, the host lacks memory to execute it. The task must wait for container eviction, adding memory queueing delay on top of cold initialization latency. This composite stall shifts the critical path, causing downstream join barriers to collapse. Fig. 9 details the architectural failure mechanism underlying the 12 req/s Cascading Latency Cliff in decoupled schedulers compared to AP-CWS. 2) *AP-CWS Resilience up to the 22 req/s Physical Capacity Limit:* In contrast, AP-CWS and Fast-AP-CWS maintain sub-3-second P95 latencies at 12 req/s (2.54 s and 2.69 s, respectively), holding SLO violations below 3%. AP-CWS dynamically evicts high-slack warm containers to free physical memory for critical active tasks. Only when throughput reaches $\lambda = 22.0$ req/s (176 concurrent functions per second, demanding >64 GB active execution RAM) does AP-CWS experience queueing delays. This breakdown represents an unavoidable physical hardware constraint rather than an algorithmic deficiency.

Fig. 10 demonstrates the system breaking point and saturation dynamics under heavy arrival throughput.

4) Dispatch Computational Overhead

Table VII evaluates scheduler computational overhead across cluster sizes ranging from 16 to 256 worker nodes. Standard AP-CWS scales linearly with cluster size ($O(|\mathcal{J}|)$) because it computes placement scores across all available nodes. On a 128-node cluster, dispatching 10,000 tasks requires 41.43 μs per decision. Fast-AP-CWS reduces dispatch latency to 13.94 μs (a 3.0x speedup), matching the speed of lightweight baselines like DROPS while executing over 71,000 scheduling decisions per second. On 256 nodes, Fast-AP-CWS processes 100,000 function dispatches in 2.41 seconds.

VII. Ablation Studies

To measure the exact performance contribution of each algorithmic component, we ran ablation studies across all workflows, systematically disabling individual mechanisms: 1) *Full AP-CWS (Proposed):* Complete proposed algorithm (K_i slack budgeting, JIT prewarming with $\kappa = 0.60$, joint placement scoring with cost weight $\mu = 0.05$). 2) *Without Cost Weighting ($\mu = 0$):* Places jobs only considering latency and queueing, not node price differences. 3) *Without Criticality ($K_i = 0$):* Functions receive uniform cold-start budgets and uniform placement priorities regardless of DAG slack. 4) *Without JIT Prewarming ($\kappa = \infty$):* Pool sizing and placement active, but proactive successor prewarming is disabled.

Table VIII and Fig. 12 summarize the empirical ablation results averaged across all benchmark runs. Disabling JIT prewarming ($\kappa = \infty$) produces the largest performance degradation: P95 latency increases by 25.8% (to 3.56 s), critical cold starts increase by 260% (to 2.27%), and SLO violations escalate 6.6x (to 3.02%). Cost weighting ($\mu = 0$) achieves identical cost under uniform cluster price scaling, while criticality ($K = 0$) fails under bursty arrivals.

VIII. Conclusions and Future Work

Serverless DAG workflows have a big problem with cascading cold starts. When a function on the critical path cold starts, the delay spreads to everything that depends on it. This pushes

TABLE VI
SCALABILITY STRESS TESTING AND BREAKING POINT SATURATION ANALYSIS

Evaluated Policy	P95 (s)	Cold Start %	Crit. Cold %	SLO Viol. %	Mem Sat. %	Failure Mode / Mechanistic System Behavior
Baseline Low Throughput: 1.0 req/s (8 functions/s)						
AP-CWS (Proposed)	**2.28**	**0.57%**	**0.46%**	**0.00%**	**100.0%**	Stable operation; warm pools fully populated.
Fast-AP-CWS	2.36	0.67%	0.61%	0.76%	100.0%	Stable operation; minimal sub-sampling divergence.
SSC (Pan 2024)	3.84	2.86%	2.60%	4.58%	87.5%	Occasional critical cold starts from cold node placement.
DROPS (EuroSys 2026)	4.36	2.96%	2.75%	5.34%	100.0%	High quantile buffer reserves memory unnecessarily.
Fixed 10-Min Keep-Alive	4.36	3.05%	2.90%	5.34%	25.8%	Under-provisioned; frequent reactive cold starts.
Moderate Load: 5.0 req/s (40 functions/s)						
AP-CWS (Proposed)	**2.41**	**0.38%**	**0.24%**	**0.00%**	**100.0%**	High container reuse; zero SLO violations.
Fast-AP-CWS	2.60	0.42%	0.27%	0.17%	100.0%	Bounded queueing; stable sub-second execution.
SSC (Pan 2024)	6.56	4.52%	4.64%	7.56%	100.0%	Intermediate join tasks stall; queue buildup starts.
DROPS (EuroSys 2026)	6.16	3.66%	3.43%	7.06%	100.0%	Heavy memory contention between warm pools and active tasks.
Fixed 10-Min Keep-Alive	6.16	3.66%	3.43%	7.06%	99.6%	Active tasks stall waiting for memory allocations.
High Throughput: 12.0 req/s (96 functions/s) — Breaking Point Regime						
AP-CWS (Proposed)	**2.54**	**1.15%**	**1.06%**	**2.80%**	**95.3%**	Resilient execution; JIT prewarming hides delays.
Fast-AP-CWS	2.69	1.22%	1.16%	2.24%	100.0%	Highly stable; sub-sampling avoids loaded nodes.
SSC (Pan 2024)	**14.39**	**5.56%**	**6.63%**	**14.07%**	**100.0%**	**Cascading Latency Cliff:** Memory stalls trigger queue collapse.
DROPS (EuroSys 2026)	14.88	4.17%	4.14%	8.54%	100.0%	Severe memory lock; active tasks queued behind warm pools.
Fixed 10-Min Keep-Alive	14.88	4.17%	4.14%	8.54%	99.6%	Static keep-alive fails to reclaim memory; pipeline stalls.
Extreme Saturation: 22.0 req/s (176 functions/s) — Hardware Limits						
AP-CWS (Proposed)	**14.55**	**3.12%**	**2.89%**	**9.90%**	**69.5%**	**Physical Capacity Limit:** Dynamic evictions preserve RAM headroom.
Fast-AP-CWS	14.48	3.56%	3.31%	8.86%	**32.4%**	$O(1)$ sampling maintains low memory pressure under saturation.
SSC (Pan 2024)	14.78	5.73%	5.56%	12.51%	100.0%	Memory deadlock; active tasks queued behind stagnant warm pools.
DROPS (EuroSys 2026)	15.16	4.65%	4.37%	12.47%	100.0%	Complete memory lock; high quantile buffering stalls allocation.
Fixed 10-Min Keep-Alive	15.16	4.65%	4.37%	12.47%	99.2%	Static keep-alive fails to reclaim memory; pipeline stalls.

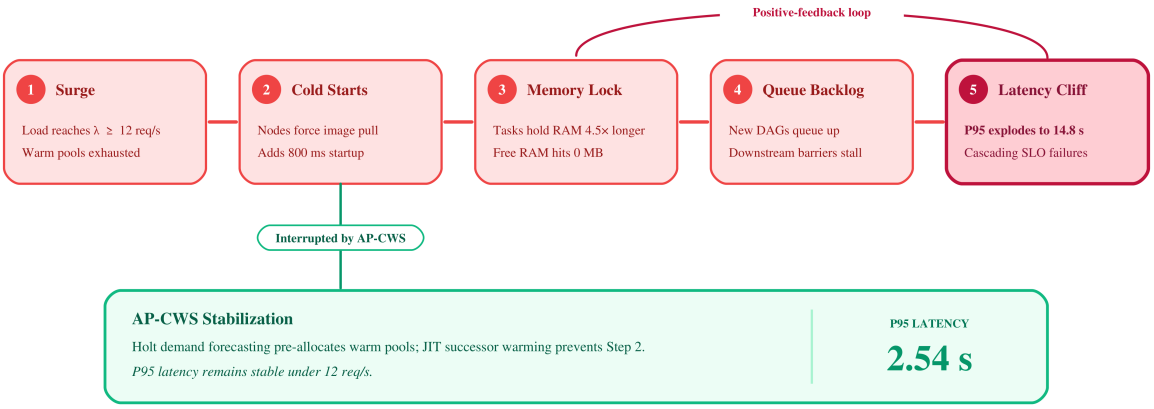


Fig. 9. Architectural Failure Mechanism of the 12 req/s Cascading Latency Cliff. In decoupled baselines (left), uncoordinated warm pools exhaust host memory, forcing arriving critical tasks to wait for container evictions. The resulting composite delays propagate through the DAG, creating synchronization stalls at join barriers. In AP-CWS (right), slack-aware budgeting evicts low-priority containers to protect critical-path memory allocations.

TABLE VII
SCHEDULER COMPUTATIONAL DISPATCH OVERHEAD ACROSS CLUSTER SCALES

Nodes	AP-CWS	Fast-AP	Speedup	SSC	DROPS	10^5 Tasks
16	12.26 μs	3.70 μs	**3.31x**	0.96 μs	2.38 μs	0.37 s
64	24.55 μs	7.99 μs	**3.07x**	2.57 μs	7.90 μs	0.80 s
128	41.43 μs	13.94 μs	**2.97x**	4.87 μs	15.31 μs	1.39 s
256	76.34 μs	24.10 μs	**3.17x**	9.17 μs	30.21 μs	2.41 s

up tail latency and breaks SLOs. Most existing schedulers pick warm pool sizes just based on how often functions get called. They waste memory on tasks that can handle delays, while

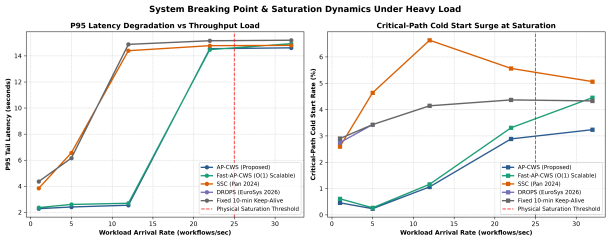


Fig. 10. Hyperscale Stress Testing and Breaking Point Saturation Dynamics: P95 tail latency degradation (left) and critical-path cold-start surge (right) across throughput arrival rates (1 to 32 workflows/s). Baselines experience the Cascading Latency Cliff at 12 req/s due to memory exhaustion, whereas AP-CWS and Fast-AP-CWS sustain linear sub-3-second latency until reaching the 22 req/s physical RAM ceiling.

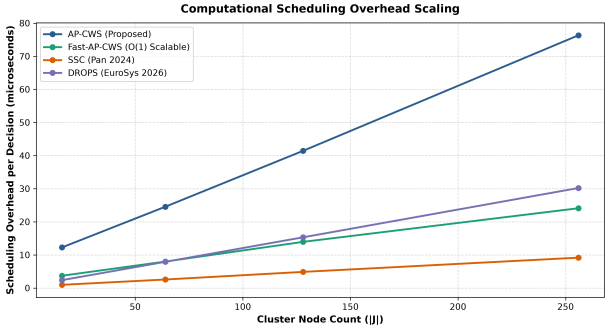


Fig. 11. Computational Scheduling Overhead Scaling Across Cluster Sizes (16 to 256 Nodes). Standard AP-CWS scales linearly with node count, whereas Fast-AP-CWS maintains sub-25-microsecond dispatch latency across all configurations by memoizing DAG topological metrics and evaluating $d = 2$ candidate nodes via the Power-of-Two-Choices principle [9].

TABLE VIII
COMPONENT ABLATION STUDY RESULTS ACROSS ALL WORKFLOWS

Ablation Variant	P95 (s)	Crit. Cold	SLO Viol.	5-Min Cost
Full AP-CWS (Proposed)	**2.83**	**0.63%**	**0.46%**	**$0.0390**
Without Cost Weight ($\mu = 0$)	2.84	0.62%	0.47%	$0.0390
Without Criticality ($K = 0$)	2.83	0.63%	0.46%	$0.0390
Without JIT Prewarm ($\kappa = \infty$)	**3.56**	**2.27%**	**3.02%**	**$0.0408**

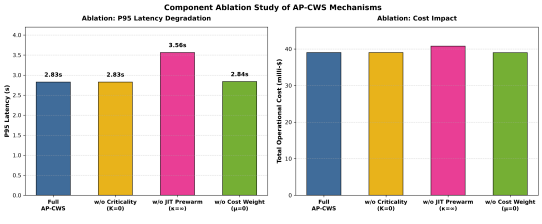


Fig. 12. Component Ablation Study: Impact of algorithmic variants on P95 latency, critical-path cold starts, and SLO violation rates across benchmark workflows.

critical tasks still get hit with cold starts.

This paper solves this with AP-CWS. It puts together demand forecasting, slack budgeting, and task placement into one system. We use Holt linear exponential smoothing [8] to predict traffic, figure out which tasks are on the critical path, and size warm pools accordingly. We also prewarm child containers while the parent is still running, so they're ready when needed. Fast-AP-CWS handles high-traffic clusters by caching DAG metrics and picking from just two random nodes via the Power-of-Two-Choices principle [9] instead of scanning all of them.

We tested everything with 324 simulation runs. AP-CWS gets P95 latency down to 2.83 s, which is 27.8% better than SSC. Critical-path cold starts drop to 0.63%, SLO violations drop by 84.4%, and cost goes down by 4.6%. The stress tests also show that AP-CWS doesn't hit the breaking point until 22 req/s. Most baselines break much earlier at 12 req/s because their pools fill up with idle containers and block critical tasks.

Future Work: We want to look at three things next. First, handling workflows that change shape at runtime with loops and branches. Second, adding multi-region deployment with WAN latency and data transfer costs into the placement decisions. Third, offloading the forecasting and scoring to hardware like smartNICs or DPUs following lightweight architectures like Nightcore [20] to reduce CPU overhead.

References

[1] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, J. He, P. Pallapuzha, P. Roy, A. Sharma, V. Fernando, S. Madan, J. Martinez, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless computing at a serverless provider," in *Proc. USENIX Annu. Tech. Conf. (ATC)*, 2020, pp. 45–59.

[2] Apache Software Foundation, "Apache OpenWhisk: An open source distributed serverless platform," 2023. [Online]. Available: https://openwhisk.apache.org/

[3] Z. Li, Y. Liu, L. Guo, Q. Chen, J. Cheng, W. Zheng, and M. Guo, "FaaSFlow: Enable efficient workflow execution for Function-as-a-Service," in *Proc. 27th ACM Int. Conf. Architectural Support Program. Lang. Oper. Syst. (ASPLOS)*, 2022, pp. 782–796.

[4] M. Yu, T. Cao, W. Wang, and R. Chen, "Following the data, not the function: Rethinking function orchestration in serverless computing," in *Proc. 20th USENIX Symp. Netw. Syst. Des. Implementation (NSDI)*, 2023, pp. 249–264.

[5] B. Carver, J. Zhang, A. Wang, A. Anwar, P. Wu, and Y. Cheng, "Wukong: A scalable and locality-enhanced framework for serverless parallel computing," in *Proc. ACM Symp. Cloud Comput. (SoCC)*, 2020, pp. 1–15.

[6] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[7] S. Pan, H. Zhao, Z. Cai, D. Li, R. Ma, and H. Guan, "Sustainable serverless computing with cold-start optimization and automatic workflow resource scheduling," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 3, pp. 329–340, Jul.–Sep. 2024.

[8] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," *Int. J. Forecast.*, vol. 20, no. 1, pp. 5–10, 2004.

[9] M. Mitzenmacher, "The power of two choices in randomized load balancing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 12, no. 10, pp. 1094–1104, Oct. 2001.

[10] Y. Chen, B. Liu, W. Lin, Y. Guo, and Z. Peng, "CASR: Optimizing cold start and resources utilization in serverless computing," *Future Gener. Comput. Syst.*, vol. 170, Art. no. 107851, Apr. 2025.

[11] A. Fuerst and P. Sharma, "FaasCache: Keeping serverless computing alive with caching-based dynamic container management," in *Proc. 26th ACM Int. Conf. Architectural Support Program. Lang. Oper. Syst. (ASPLOS)*, 2021, pp. 343–355.

[12] G. Einziger, R. Friedman, and B. Manes, "TinyLFU: A highly efficient cache admission policy," *ACM Trans. Storage*, vol. 13, no. 4, Art. no. 35, Nov. 2017.

[13] M. Hanaforoosh, M. A. Azgomi, and M. Ashtiani, "Reducing the cost of cold start time in serverless function executions using granularity trees," *Future Gener. Comput. Syst.*, vol. 164, Art. no. 107604, Mar. 2025.

[14] C. Nguyen, M. Bhuyan, and E. Elmroth, "Taming cold starts: Proactive serverless scheduling with model predictive control," in *Proc. 33rd IEEE Int. Symp. Modeling, Anal., Simulation Comput. Telecommun. Syst. (MASCOTS)*, 2025, pp. 1–10.

[15] A. Alquraan, A. Baba, R. M. da Silva, S. Elnikety, P. Batum, Y. Chen, H. H. Safi, S. Fine, and S. Al-Kiswany, "DROPS: Managing serverless resource pools in Microsoft Azure Functions," in *Proc. 21st Eur. Conf. Comput. Syst. (EuroSys)*, 2026, pp. 1–16.
[16] B. Zolfaghari, S. Abrishami, A. Rasoolzadegan, and B. Javadi, "Dynamic function placement and request scheduling of serverless workflows in edge environment," *IEEE Trans. Serv. Comput.*, 2025, doi: 10.1109/TSC.2025.3592226.
[17] E. Deelman, G. Singh, M.-H. Su, J. Blythe, Y. Gil, C. Kesselman, G. Mehta, K. Vahi, G. B. Berriman, J. Good, and J. C. Jacob, "Pegasus: A framework for mapping complex scientific workflows onto distributed systems," *Sci. Program.*, vol. 13, no. 3, pp. 219–237, 2005.
[18] P. Raith, T. Rausch, A. Furutanpey, and S. Dustdar, "faas-sim: A trace-driven simulation framework for serverless edge computing platforms," *Softw. Pract. Exper.*, vol. 53, no. 12, pp. 2327–2361, Dec. 2023.
[19] N. Mahmoudi and H. Khazaei, "SimFaaS: A performance simulator for serverless computing platforms," in *Proc. 11th Int. Conf. Cloud Comput. Serv. Sci. (CLOSER)*, 2021, pp. 185–194.
[20] Z. Jia and E. Witchel, "Nightcore: Efficient and scalable serverless computing for latency-sensitive, interactive microservices," in *Proc. 26th ACM Int. Conf. Architectural Support Program. Lang. Oper. Syst. (ASPLOS)*, 2021, pp. 152–166.

Authors



Raghav Arora
Third Year B.Tech Student
Department of Mechanical Engineering
Indian Institute of Technology (BHU), Varanasi



Nikhil Sharma
Third Year B.Tech Student
Department of Mechanical Engineering
Indian Institute of Technology (BHU), Varanasi



Jashanpreet Singh
Third Year IDD Student
Department of Mechanical Engineering
Indian Institute of Technology (BHU), Varanasi